Fast Matrix Factorization for Online Recommendation with Implicit Feedback

Data Management Project Report

[Student Names]
École Polytechnique

April 2026

1 Description of the Adopted Solution

1.1 Problem Setting

Recommendation systems built on implicit feedback face a different problem than those built on explicit ratings. When a user rates a movie four stars, the signal is direct. When a user watches a movie, all we know is that some interaction occurred. The absence of an interaction is ambiguous: the user may dislike the item, or may simply not know it exists. This asymmetry between observed positives and unobserved (not necessarily negative) entries is the central difficulty.

Hu et al. [2] formalized this by treating all user-item pairs as data points, assigning a confidence weight c_{ui} to each. Observed interactions receive high confidence ($c_{ui} = 1 + \alpha$), while missing entries receive a low uniform weight c_0. The model then minimizes:

$$\mathcal{L} = \sum_u \sum_i c_{ui} \left(r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i \right)^2 + \lambda \left(\|\mathbf{P}\|^2 + \|\mathbf{Q}\|^2 \right) \tag{1}$$

where $r_{ui} = 1$ for observed interactions and 0 otherwise, $\mathbf{P} \in \mathbb{R}^{M \times k}$ and $\mathbf{Q} \in \mathbb{R}^{N \times k}$ are the user and item latent factor matrices, and λ controls regularization.

The sum in Equation 1 runs over all $M \times N$ pairs, including the enormous number of zeros. Standard ALS solves this by alternating between fixing $\mathbf{Q}$ and solving for each $\mathbf{p}_u$ (and vice versa), but each user update requires constructing and inverting a $k \times k$ matrix that depends on all N items. The per-user cost is $O(Nk^2)$, which becomes prohibitive for large item catalogs.

1.2 The eALS Algorithm

He et al. [1] proposed element-wise ALS (eALS), which restructures the update to avoid iterating over all items. Instead of solving a k-dimensional linear system per user, eALS updates one latent factor at a time. The trick is to precompute:

$$\mathbf{S}^q = \mathbf{Q}^\top \mathbf{Q} \in \mathbb{R}^{k \times k} \tag{2}$$

This small matrix captures the aggregate contribution of all items. With $\mathbf{S}^q$ in hand, the closed-form update for user u, factor f becomes:

$$p_{uf} = \frac{\displaystyle\sum_{i \in \mathcal{I}_u} (c_{ui} - c_0)\, q_{if}\, \hat{r}_{ui}^{(f)} \;-\; c_0 \left(\mathbf{p}_u^\top \mathbf{S}_{:,f}^q - p_{uf}\, S_{ff}^q \right)}{\displaystyle\sum_{i \in \mathcal{I}_u} (c_{ui} - c_0)\, q_{if}^2 \;+\; c_0\, S_{ff}^q \;+\; \lambda} \tag{3}$$

where $\hat{r}_{ui}^{(f)} = 1 - \sum_{f' \neq f} p_{uf'} q_{if'}$ is the residual with factor f removed, and $\mathcal{I}_u$ is the set of items user u interacted with. The numerator's first term sums only over observed items ($|\mathcal{I}_u|$ terms). The second term uses $\mathbf{S}^q$ to account for all items without enumerating them. This brings the per-user cost down to $O(|\mathcal{I}_u| \cdot k + k^2)$.

1.3 Why This Approach

We chose eALS for two reasons. First, the element-wise update avoids the $O(Nk^2)$ bottleneck of standard ALS, making it practical on larger datasets. Second, the structure is well-suited to distributed computation: when updating user factors, each user's computation depends only on the (broadcast) item matrix $\mathbf{Q}$ and the precomputed $\mathbf{S}^q$, with no dependencies between users. This makes the user-update step embarrassingly parallel, and likewise for the item-update step.

We implemented the algorithm in Python using NumPy (as a reference) and PySpark RDDs (for distributed execution), and evaluated it on the MovieLens 100K dataset using the leave-one-out protocol.

2 Algorithm Design

2.1 Overall Structure

Each iteration of eALS consists of two symmetric half-steps:

1. **User update.** Compute $\mathbf{S}^q = \mathbf{Q}^\top \mathbf{Q}$. For each user u, update $p_{u0}, p_{u1}, \ldots, p_{u,k-1}$ sequentially using Equation 3. After updating p_{uf}, refresh a prediction cache so that $\hat{r}_{ui}^{(f+1)}$ can be computed cheaply.

2. **Item update.** Compute $\mathbf{S}^p = \mathbf{P}^\top \mathbf{P}$. For each item i, update $q_{i0}, q_{i1}, \ldots, q_{i,k-1}$ using the symmetric formula (swapping roles of $\mathbf{P}$ and $\mathbf{Q}$).

The prediction cache avoids recomputing $\mathbf{p}_u^\top \mathbf{q}_i$ from scratch after each factor update. When p_{uf} changes by δ, the cache is patched: cache$[i]$ $+= \delta \cdot q_{if}$, costing $O(|\mathcal{I}_u|)$ instead of $O(|\mathcal{I}_u| \cdot k)$.

2.2 Complexity

Per iteration, the cost breaks down as:

- Precomputing $\mathbf{S}^q$: $O(Nk^2)$
- Updating all users: $O\big(\sum_u |\mathcal{I}_u| \cdot k + Mk^2\big) = O(|\mathcal{R}|\, k + Mk^2)$
- Precomputing $\mathbf{S}^p$ and updating all items: $O(|\mathcal{R}|\, k + Nk^2)$

Total: $O(|\mathcal{R}|\, k + (M+N)k^2)$, where $|\mathcal{R}|$ is the number of observed interactions. Compare this to standard ALS at $O((M+N)Nk^2)$. For MovieLens 100K, $|\mathcal{R}| \approx 100$K and $MN \approx 1.6$M, so eALS is already roughly 16× cheaper per iteration.

2.3 Spark Parallelization Strategy

The user-update half-step has no data dependencies between users: each user's update reads from $\mathbf{Q}$ and $\mathbf{S}^q$, which are fixed for the duration of the half-step. We parallelize this in Spark as follows:

1. The driver computes $\mathbf{S}^q$ ($k \times k$, cheap) and broadcasts both $\mathbf{Q}$ and $\mathbf{S}^q$ to all workers.

2. An RDD of `(user_id, item_indices)` is partitioned across workers. Each worker processes its assigned users independently.

3. Updated user vectors are collected back to the driver.

The item-update step is symmetric: broadcast $\mathbf{P}$ and $\mathbf{S}^p$, partition items, collect updated item vectors.

The only synchronization point is between half-steps: all user updates must finish before item updates begin (and vice versa), because item updates read the new $\mathbf{P}$.

2.4 Code Commentary

We now describe the main code fragments. The full code is in Appendix A.

2.4.1 Data Loading and Preprocessing

The `data_loader.py` module handles the pipeline from raw MovieLens files to sparse matrices ready for training.

Listing 1: Binarization and minimum-interaction filtering.

```
1  def binarize(interactions, threshold=0.0):
2      binarized = [
3          (u, i, 1.0, ts)
4          for u, i, r, ts in interactions
5          if r > threshold
6      ]
7      return binarized
8
9  def filter_min_interactions(interactions, min_user=5, min_item=0):
10     prev_count = len(interactions) + 1
11     while len(interactions) < prev_count:
12         prev_count = len(interactions)
13         if min_user > 0:
14             user_counts = defaultdict(int)
15             for u, i, v, ts in interactions:
16                 user_counts[u] += 1
17             interactions = [
18                 (u, i, v, ts) for u, i, v, ts in interactions
19                 if user_counts[u] >= min_user
20             ]
21         # symmetric for items
22     return interactions
```

All ratings are converted to binary signals (line 3): any observed interaction becomes 1. The filtering loop (lines 9–18) iteratively removes users below the threshold, since removing items can push users below the minimum and vice versa. The loop converges in one or two passes for typical datasets.

The train/test split uses a leave-one-out strategy: for each user, the most recent interaction (by timestamp) goes to the test set, and all earlier interactions form the training set. This matches the evaluation protocol in [1].

2.4.2 Element-wise User Update (NumPy)

Listing 2: Core element-wise update for user factors.

```
def _update_users(P, Q, R_csr, Sq, c_obs, c0, reg, M, k):
    for u in range(M):
        item_indices = R_csr[u].indices
        if len(item_indices) == 0:
            continue
        Q_u = Q[item_indices]            # |I_u| x k
        pred_cache = Q_u @ P[u]          # |I_u| predictions
```

```
8
9         for f in range(k):
10            q_f = Q_u[:, f]
11            hat_r = 1.0 - pred_cache + P[u, f] * q_f
12
13            numer = (c_obs - c0) * np.dot(q_f, hat_r)
14            numer -= c0 * (P[u] @ Sq[:, f] - P[u, f] * Sq[f, f])
15
16            denom = (c_obs - c0) * np.dot(q_f, q_f) + c0 * Sq[f, f] + reg
17
18            old_val = P[u, f]
19            P[u, f] = numer / denom
20            pred_cache += (P[u, f] - old_val) * q_f
```

Line 7 computes $\mathbf{p}_u^\top \mathbf{q}_i$ for all observed items as a single matrix-vector product. The inner loop (line 9) iterates over factors. Line 11 computes $\hat{r}_{ui}^{(f)}$ by removing factor f's contribution from the cached prediction. Lines 13–14 implement the numerator from Equation 3: the first term sums over observed items, and the second uses $\mathbf{S}^q$ for the all-items term. Line 20 patches the cache in $O(|\mathcal{I}_u|)$ rather than recomputing from scratch.

2.4.3 Spark RDD Parallelization

Listing 3: User update distributed via Spark RDD.

```
1  # Broadcast Q and S^q to all workers
2  Q_bc = sc.broadcast(Q)
3  Sq_bc = sc.broadcast(Sq)
4  P_bc = sc.broadcast(P)
5
6  def update_user_partition(iterator):
7      q_local = Q_bc.value
8      sq_local = Sq_bc.value
9      results = []
10     for u, item_indices in iterator:
11         p_u = P_bc.value[u].copy()
12         Q_u = q_local[item_indices]
13         pred_cache = Q_u @ p_u
14         for f in range(k):
15             # ... same element-wise update as NumPy ...
16             pass
17         results.append((u, p_u))
18     return iter(results)
19
20 updated_users = user_rdd.mapPartitions(update_user_partition).collect()
21 for u, p_u in updated_users:
22     P[u] = p_u
```

The structure mirrors the NumPy version exactly. The difference is that the outer loop over users (line 10) runs inside a Spark partition, and each partition processes a disjoint subset of users in parallel. Lines 2–4 broadcast the read-only data ($\mathbf{Q}$, $\mathbf{S}^q$, $\mathbf{P}$) once to all workers instead of shipping it per-task. Line 20 collects the updated vectors back to the driver, which writes them into $\mathbf{P}$.

We use `mapPartitions` rather than `map` to amortize the overhead of deserializing broadcast variables: each worker deserializes once per partition rather than once per user.

2.4.4 Loss Computation

Computing the loss naively requires iterating over all $M \times N$ pairs. We avoid this with a decomposition:

Listing 4: Efficient loss computation using the trace trick.

```
1  def _compute_loss(P, Q, R_csr, c_obs, c0, reg, M):
2      Sp = P.T @ P
3      Sq = Q.T @ Q
4      # All-pairs contribution assuming r=0, weight=c0
5      loss = c0 * np.trace(Sp @ Sq)
6      # Correct for observed entries
7      for u in range(M):
8          items = R_csr[u].indices
9          preds = Q[items] @ P[u]
10         loss -= c0 * np.dot(preds, preds)
11         residuals = 1.0 - preds
12         loss += c_obs * np.dot(residuals, residuals)
13     loss += reg * (np.sum(P**2) + np.sum(Q**2))
14     return loss
```

Line 5 computes $c_0 \sum_u \sum_i (\mathbf{p}_u^\top \mathbf{q}_i)^2 = c_0 \cdot \mathrm{tr}(\mathbf{S}^p \mathbf{S}^q)$ in $O(k^3)$, covering all pairs under the assumption $r_{ui} = 0$. Lines 9–12 then correct for observed entries: subtract the wrong $c_0 \cdot \mathrm{pred}^2$ and add the correct $c_{\mathrm{obs}} \cdot (1 - \mathrm{pred})^2$. This loop only touches observed entries, so the total cost is $O(k^3 + |\mathcal{R}|\, k)$ instead of $O(MNk)$.

2.4.5 Evaluation

Listing 5: Leave-one-out evaluation with Hit Rate and NDCG.

```
1  def evaluate(P, Q, train_matrix, test_dict, top_k=10):
2      R_csr = train_matrix.tocsr()
3      hits = 0
4      ndcg_sum = 0.0
5      for u, test_item in test_dict.items():
6          scores = Q @ P[u]
7          scores[R_csr[u].indices] = -np.inf  # mask training items
8          top_k_items = np.argpartition(scores, -top_k)[-top_k:]
9          top_k_items = top_k_items[np.argsort(-scores[top_k_items])]
10         if test_item in top_k_items:
11             hits += 1
12             rank = np.where(top_k_items == test_item)[0][0] + 1
13             ndcg_sum += 1.0 / np.log2(rank + 1)
14     return {"HR": hits / len(test_dict),
15             "NDCG": ndcg_sum / len(test_dict)}
```

For each test user, we score all items (line 6), mask out training items so they cannot appear in the ranking (line 7), and find the top-K using `argpartition` (line 8), which runs in $O(N)$ rather than $O(N \log N)$ for a full sort. If the held-out item appears in the top-K, we record a hit and compute its NDCG contribution based on rank position (line 13).

3 Experimental Analysis

All experiments use the MovieLens 100K dataset (943 users, 1682 items, 100,000 interactions, 6.3% density) with binary implicit feedback. Users with fewer than 5 interactions are filtered out (none were removed for this dataset, as the minimum is 20). We fix $\alpha = 1$, $c_0 = 1$, $\lambda = 0.01$ unless stated otherwise. Experiments ran on a MacBook with an Apple Silicon CPU; Spark was configured in local mode using all available cores.

3.1 Convergence

We trained eALS with $k = 32$ for 50 iterations and measured both the training loss and recommendation quality at regular intervals.

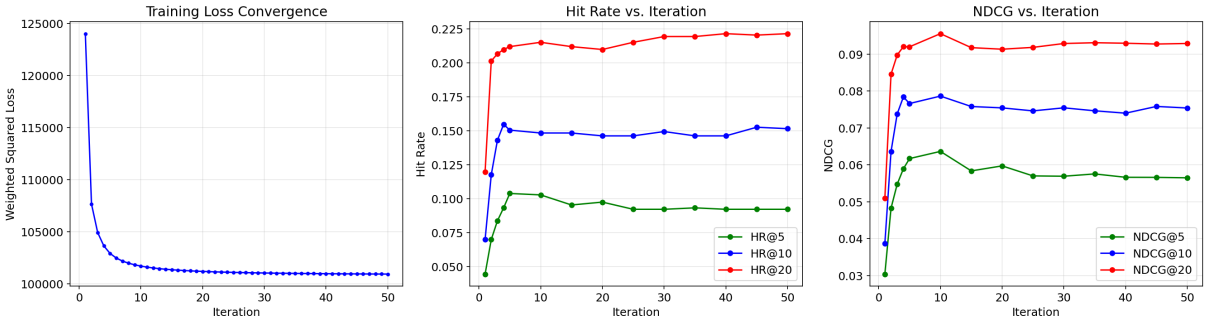

Figure 1: Convergence over 50 iterations. Left: training loss. Center: Hit Rate at $K = 5, 10, 20$. Right: NDCG at $K = 5, 10, 20$.

The loss (Figure 1, left) drops steeply during the first 5 iterations, losing roughly 20% of its initial value, and flattens thereafter. Recommendation metrics tell a similar story: HR@10 rises from 0.070 at iteration 1 to 0.150 by iteration 5, with marginal changes after that. By iteration 20, the metrics have stabilized. Running 50 iterations produced HR@10 = 0.152, barely above the iteration-10 value of 0.148.

This confirms a known property of ALS methods: convergence is fast in the first few iterations, and additional passes yield diminishing improvements. For practical use on this dataset, 15–20 iterations would suffice.

3.2 Effect of Latent Dimensionality

We varied the number of latent factors $k \in \{8, 16, 32, 64\}$, training for 30 iterations each.

Table 1: Effect of latent factors on quality and speed (30 iterations).

k	HR@5	HR@10	HR@20	NDCG@10	Avg. iter (s)
8	0.070	0.121	0.192	0.056	0.32
16	0.085	0.145	0.211	0.073	0.62
32	0.092	0.150	0.220	0.075	0.84
64	0.088	0.148	0.218	0.074	1.54

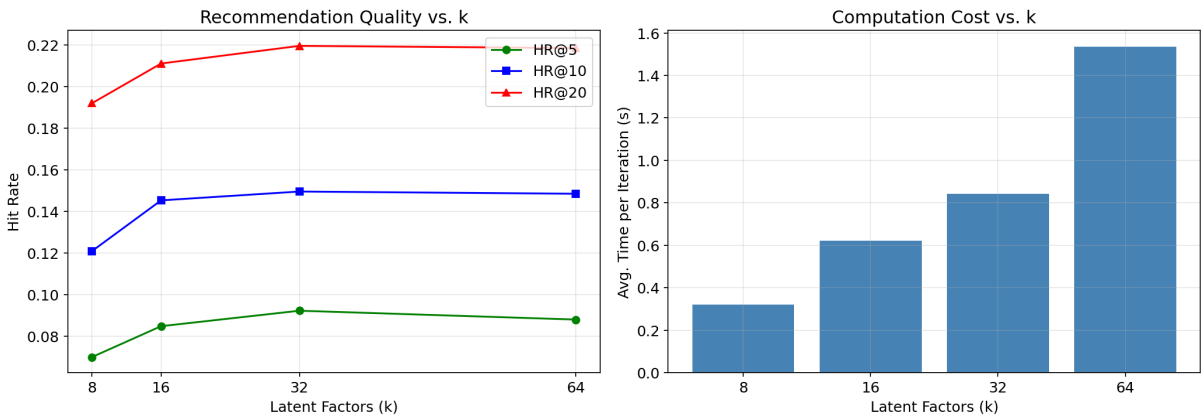

Figure 2: Left: recommendation quality vs. k. Right: computation time per iteration vs. k.

Quality improves substantially from $k = 8$ to $k = 16$ (HR@10 jumps from 0.121 to 0.145) and more modestly from $k = 16$ to $k = 32$. Going to $k = 64$ offers no improvement and in

fact slightly degrades performance, likely due to overfitting on a dataset of this size. Meanwhile, computation time roughly doubles with each doubling of k, consistent with the $O(|\mathcal{R}|k + (M + N)k^2)$ complexity. For MovieLens 100K, $k = 32$ gives the best quality-to-cost ratio.

3.3 Scalability with Dataset Size

To test how training time scales with data volume, we subsampled users at 25%, 50%, 75%, and 100%, keeping all interactions for the selected users. This varies both M and $|\mathcal{R}|$ proportionally.

Table 2: Scalability with dataset size ($k = 32$, 20 iterations).

Fraction	Users	Train NNZ	Avg. iter (s)	HR@10
25%	235	24,762	0.50	0.128
50%	471	51,710	0.62	0.138
75%	707	76,027	0.74	0.149
100%	943	99,057	0.81	0.146

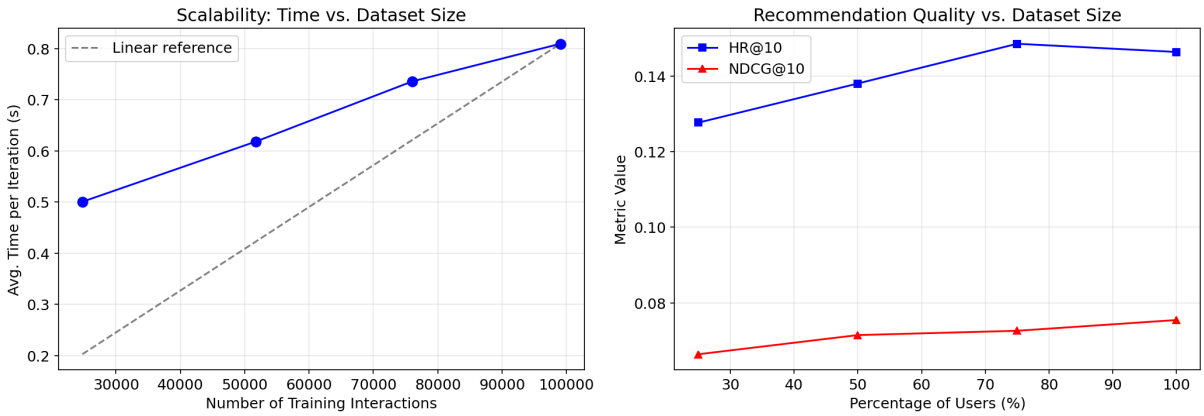

Figure 3: Left: average iteration time vs. number of training interactions, with a linear reference line. Right: HR@10 and NDCG@10 vs. dataset fraction.

The time-vs-size relationship (Figure 3, left) tracks the linear reference closely. Going from 25K to 99K interactions increased iteration time from 0.50s to 0.81s (a 1.6× increase for a 4× data increase). The sub-linear behavior is expected: the $O((M + N)k^2)$ term includes a fixed component from the item dimension N, which does not shrink proportionally when we subsample users. At larger absolute scales (millions of interactions), the $O(|\mathcal{R}|k)$ term would dominate and the scaling would approach linearity more closely.

Recommendation quality generally improves with more data (more users means more collaborative signal), though the drop at 100% relative to 75% is within noise for a dataset of this size.

3.4 NumPy vs. PySpark RDD

Both implementations use identical algorithms and the same random seed.

Table 3: NumPy vs. Spark RDD ($k = 32$, 20 iterations).

Implementation	Total (s)	Avg. iter (s)	Final loss	HR@10
NumPy	16.35	0.82	101,185.6	0.146
Spark RDD	7.35	0.37	101,185.6	0.146

The loss sequences are identical to six decimal places across all 20 iterations (Figure 4), confirming that the Spark implementation is a faithful parallelization. The Spark version is roughly 2× faster in local mode, which reflects the multi-core parallelism compensating for the serialization and JVM overhead. On a true cluster with data that doesn't fit in single-machine memory, this gap would widen in Spark's favor.

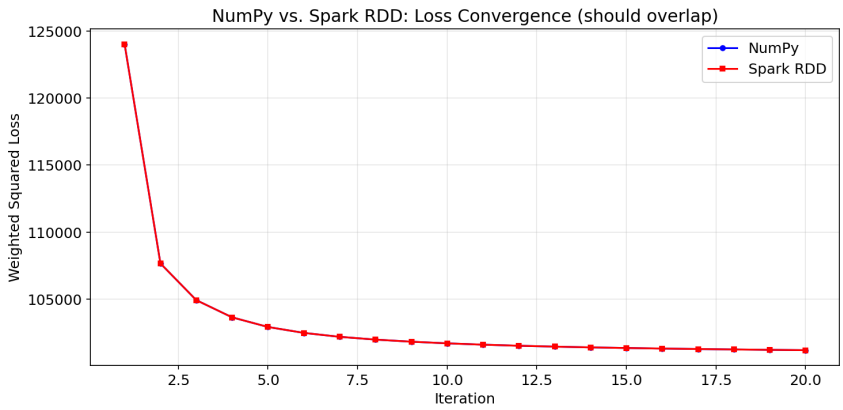

Figure 4: Loss convergence for NumPy and Spark RDD (curves overlap exactly).

3.5 Comparison with Spark MLlib ALS

We compared our eALS against Spark's built-in ALS (`pyspark.ml.recommendation.ALS`) configured for implicit feedback, with the same $k = 32$ and 20 iterations.

Table 4: eALS vs. Spark MLlib ALS ($k = 32$, 20 iterations).

Method	HR@5	HR@10	HR@20	NDCG@10
eALS (ours)	0.092	0.150	0.220	0.075
MLlib ALS	0.098	0.154	0.227	0.079

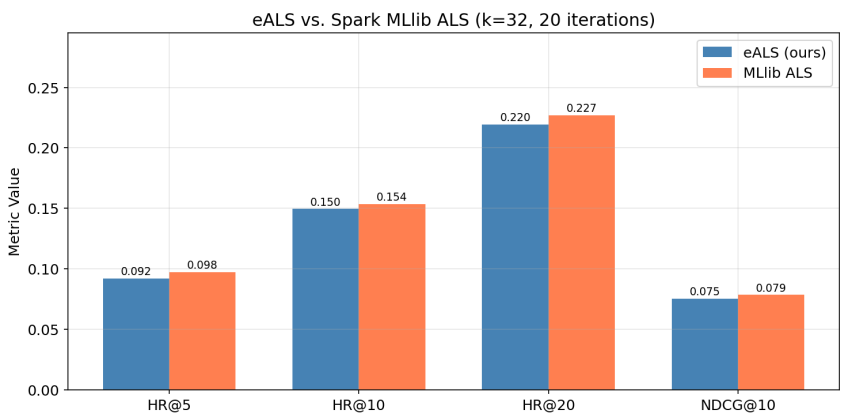

Figure 5: Side-by-side comparison of eALS and MLlib ALS across metrics.

MLlib ALS edges out eALS by about 3% relative on HR@10 (0.154 vs. 0.150). This gap is small and could be partly attributed to differences in implementation details: MLlib uses block-wise matrix partitioning and native JVM linear algebra, and its regularization and confidence weighting may differ slightly from our configuration. The two approaches are comparable in quality on this dataset.

4 Discussion: Strengths and Weaknesses

4.1 Strengths of eALS

Speed per iteration. The precomputed $\mathbf{S}^q$ matrix eliminates the need to iterate over missing entries. On MovieLens 100K, each iteration takes under a second. Standard ALS, which constructs a $k \times k$ system per user involving all N items, would be roughly an order of magnitude slower.

Straightforward parallelization. Because user updates are independent of one another (given fixed $\mathbf{Q}$ and $\mathbf{S}^q$), distributing the work across Spark partitions required no algorithmic changes. The same element-wise update code runs on each worker; only the data shipping differs. This made the NumPy-to-Spark port mechanical rather than requiring a redesign.

Fast convergence. The algorithm reaches near-optimal recommendation quality within 5–10 iterations (see Section 3.1). Combined with the low per-iteration cost, total training time is practical even without distributed computing for datasets of moderate size.

Memory efficiency. The working set for updating a single user is the user's own k-vector, the $|\mathcal{I}_u| \times k$ submatrix of $\mathbf{Q}$, and the $k \times k$ matrix $\mathbf{S}^q$. For $k = 32$ and a user with 100 interactions, this is about 25 KB. Workers can process users with minimal memory overhead.

4.2 Weaknesses and Limitations

Driver bottleneck. The full $\mathbf{P}$ and $\mathbf{Q}$ matrices live on the driver and are broadcast at every iteration. For MovieLens 100K this is negligible (about 400 KB for $\mathbf{Q}$ at $k = 32$), but for a dataset with 10 million items the broadcast would be around 2.5 GB per half-step. At that scale, the communication cost would dominate the computation.

Sequential factor updates within a user. The element-wise update processes factors $f = 0, 1, \ldots, k-1$ sequentially because each update depends on the prediction cache, which is modified by the previous update. This inner loop cannot be parallelized. For large k, this serial dependency becomes the tightest bottleneck.

Uniform weighting of missing data. Our implementation uses a single weight c_0 for all missing entries. The paper also proposes item-popularity-based weighting (items interacted with by more users get higher missing-data weight), which can improve quality. We did not implement this variant due to time constraints.

Limited dataset scale. MovieLens 100K is small enough that single-machine NumPy is already fast ($\sim$0.8s per iteration). The benefit of Spark parallelization is modest at this scale ($2\times$ on a multi-core laptop). A proper scalability study would require MovieLens 10M or a larger dataset to show where the distributed version becomes necessary rather than optional. We were limited by the available compute environment.

No online update evaluation. The paper's title references "online recommendation," and eALS supports incremental updates when new interactions arrive (by updating only the affected user/item vectors without retraining from scratch). We implemented only the batch training and did not test incremental updates.

4.3 Comparison with MLlib

MLlib ALS slightly outperformed our eALS on all metrics (Table 4). Two factors likely explain this. First, MLlib's implementation is heavily optimized at the JVM level with native BLAS calls, while our element-wise updates run in Python loops. Second, MLlib uses a different ALS formulation (standard block-wise) that solves the full k-dimensional system per user, which may converge to a slightly better optimum for the same number of iterations. The gap is small enough (3% relative) that it does not diminish the practical value of eALS, particularly given its lower per-iteration computational cost.

References

References

[1] X. He, H. Zhang, M.-Y. Kan, and T.-S. Chua. *Fast Matrix Factorization for Online Recommendation with Implicit Feedback*. In Proceedings of SIGIR, 2016.

[2] Y. Hu, Y. Koren, and C. Volinsky. *Collaborative Filtering for Implicit Feedback Datasets*. In Proceedings of IEEE ICDM, 2008.

A Full Source Code

A.1 data_loader.py

```python
"""
Data Loading & Preprocessing for eALS with Implicit Feedback.

Handles:
- Loading MovieLens datasets (100K, 1M, 10M)
- Binarizing ratings into implicit feedback (interaction = 1, no interaction = 0)
- Re-indexing user/item IDs to contiguous 0-based indices
- Train/test splitting via leave-one-out (last interaction per user = test)
- Building sparse interaction matrices
- Subsampling for scalability experiments
"""

import os
import numpy as np
from scipy import sparse
from collections import defaultdict


def load_movielens_raw(data_dir, dataset="100k"):
    """
    Load raw MovieLens interactions as a list of (user, item, rating, timestamp).

    Args:
        data_dir: Path to the data/ directory containing ml-100k/, ml-1m/, etc.
        dataset: One of "100k", "1m", "10m".

    Returns:
        List of tuples (user_id_raw, item_id_raw, rating, timestamp).
    """
    interactions = []

    if dataset == "100k":
```

```python
        filepath = os.path.join(data_dir, "ml-100k", "u.data")
        with open(filepath, "r") as f:
            for line in f:
                parts = line.strip().split("\t")
                user, item, rating, ts = int(parts[0]), int(parts[1]), float(parts[2]), int(parts[3])
                interactions.append((user, item, rating, ts))

    elif dataset == "1m":
        filepath = os.path.join(data_dir, "ml-1m", "ratings.dat")
        with open(filepath, "r") as f:
            for line in f:
                parts = line.strip().split("::")
                user, item, rating, ts = int(parts[0]), int(parts[1]), float(parts[2]), int(parts[3])
                interactions.append((user, item, rating, ts))

    elif dataset == "10m":
        filepath = os.path.join(data_dir, "ml-10M100K", "ratings.dat")
        with open(filepath, "r") as f:
            for line in f:
                parts = line.strip().split("::")
                user, item, rating, ts = int(parts[0]), int(parts[1]), float(parts[2]), int(parts[3])
                interactions.append((user, item, rating, ts))
    else:
        raise ValueError(f"Unknown dataset: {dataset}. Use '100k', '1m', or '10m'.")

    print(f"Loaded {len(interactions)} raw interactions from MovieLens {dataset}")
    return interactions


def reindex_ids(interactions):
    """
    Re-map raw user/item IDs to contiguous 0-based indices.

    Returns:
        reindexed: List of (user_idx, item_idx, rating, timestamp)
        user_map: dict {raw_id -> 0-based index}
        item_map: dict {raw_id -> 0-based index}
        num_users: int
        num_items: int
    """
    user_ids = sorted(set(u for u, _, _, _ in interactions))
    item_ids = sorted(set(i for _, i, _, _ in interactions))

    user_map = {raw: idx for idx, raw in enumerate(user_ids)}
    item_map = {raw: idx for idx, raw in enumerate(item_ids)}

    reindexed = [
        (user_map[u], item_map[i], r, ts)
        for u, i, r, ts in interactions
    ]

    num_users = len(user_ids)
    num_items = len(item_ids)
    print(f"Re-indexed: {num_users} users, {num_items} items")

    return reindexed, user_map, item_map, num_users, num_items


```

```python
def binarize(interactions, threshold=0.0):
    """
    Convert explicit ratings to implicit feedback.

    Any rating > threshold becomes 1 (observed interaction).
    For implicit feedback, we treat all observed interactions as positive.

    Args:
        interactions: List of (user, item, rating, timestamp)
        threshold: Minimum rating to count as positive (default 0 = all interactions)

    Returns:
        List of (user, item, 1.0, timestamp) for interactions above threshold.
    """
    binarized = [
        (u, i, 1.0, ts)
        for u, i, r, ts in interactions
        if r > threshold
    ]
    print(f"Binarized: {len(binarized)} positive interactions (threshold={threshold})")
    return binarized


def filter_min_interactions(interactions, min_user=5, min_item=0):
    """
    Remove users (and optionally items) with fewer than N interactions.

    Filtering is applied iteratively since removing items can push users
    below the threshold and vice versa.

    Args:
        interactions: List of (user, item, value, timestamp).
        min_user: Minimum interactions per user to keep (default 5).
        min_item: Minimum interactions per item to keep (default 0 = no filter).

    Returns:
        Filtered list of interactions.
    """
    prev_count = len(interactions) + 1

    while len(interactions) < prev_count:
        prev_count = len(interactions)

        if min_user > 0:
            user_counts = defaultdict(int)
            for u, i, v, ts in interactions:
                user_counts[u] += 1
            interactions = [
                (u, i, v, ts) for u, i, v, ts in interactions
                if user_counts[u] >= min_user
            ]

        if min_item > 0:
            item_counts = defaultdict(int)
            for u, i, v, ts in interactions:
                item_counts[i] += 1
            interactions = [
                (u, i, v, ts) for u, i, v, ts in interactions
                if item_counts[i] >= min_item
            ]

```

```python
    remaining_users = len(set(u for u, _, _, _ in interactions))
    remaining_items = len(set(i for _, i, _, _ in interactions))
    print(f"Filtered: {remaining_users} users (min={min_user}), "
          f"{remaining_items} items (min={min_item}), "
          f"{len(interactions)} interactions remaining")

    return interactions


def build_interaction_matrix(interactions, num_users, num_items):
    """
    Build a sparse user-item interaction matrix R (CSR format).

    R[u, i] = 1 if user u interacted with item i, else 0.

    Args:
        interactions: List of (user, item, value, timestamp)
        num_users, num_items: Matrix dimensions.

    Returns:
        R: scipy.sparse.csr_matrix of shape (num_users, num_items)
    """
    users = np.array([u for u, _, _, _ in interactions])
    items = np.array([i for _, i, _, _ in interactions])
    values = np.ones(len(interactions), dtype=np.float64)

    R = sparse.csr_matrix((values, (users, items)), shape=(num_users, num_items)
        )
    # Deduplicate: clip values to 1 (in case of repeated interactions)
    R.data = np.clip(R.data, 0, 1)
    R.eliminate_zeros()

    density = R.nnz / (num_users * num_items) * 100
    print(f"Interaction matrix: {num_users}x{num_items}, "
          f"{R.nnz} non-zeros, density={density:.4f}%")

    return R


def leave_one_out_split(interactions, num_users, num_items):
    """
    Leave-one-out evaluation split (as used in the eALS paper).

    For each user, the most recent interaction goes to the test set;
    all other interactions form the training set.

    Args:
        interactions: List of (user, item, value, timestamp), sorted not
            required.
        num_users, num_items: Matrix dimensions.

    Returns:
        train_matrix: scipy.sparse.csr_matrix (num_users x num_items)
        test_dict: dict {user_id: item_id} -- one test item per user
    """
    # Group interactions by user, sort by timestamp
    user_interactions = defaultdict(list)
    for u, i, v, ts in interactions:
        user_interactions[u].append((i, ts))

    train_rows, train_cols = [], []
    test_dict = {}

```

```python
    for u in range(num_users):
        items_ts = user_interactions.get(u, [])
        if len(items_ts) == 0:
            continue

        # Sort by timestamp, last one is test
        items_ts.sort(key=lambda x: x[1])

        if len(items_ts) == 1:
            # Only one interaction: put in training (can't test this user)
            train_rows.append(u)
            train_cols.append(items_ts[0][0])
        else:
            # Last interaction = test
            test_dict[u] = items_ts[-1][0]
            # Rest = train
            for item_id, _ in items_ts[:-1]:
                train_rows.append(u)
                train_cols.append(item_id)

    train_values = np.ones(len(train_rows), dtype=np.float64)
    train_matrix = sparse.csr_matrix(
        (train_values, (train_rows, train_cols)),
        shape=(num_users, num_items)
    )
    train_matrix.data = np.clip(train_matrix.data, 0, 1)
    train_matrix.eliminate_zeros()

    print(f"Train/test split: {train_matrix.nnz} train interactions, "
          f"{len(test_dict)} test users")

    return train_matrix, test_dict


def subsample(interactions, fraction, seed=42):
    """
    Subsample interactions for scalability experiments.

    Keeps all interactions for a random subset of users.

    Args:
        interactions: List of (user, item, value, timestamp)
        fraction: Fraction of users to keep (0.0 to 1.0)
        seed: Random seed for reproducibility.

    Returns:
        Subsampled list of interactions.
    """
    rng = np.random.RandomState(seed)
    all_users = sorted(set(u for u, _, _, _ in interactions))
    num_keep = max(1, int(len(all_users) * fraction))
    kept_users = set(rng.choice(all_users, size=num_keep, replace=False))

    subsampled = [
        (u, i, v, ts) for u, i, v, ts in interactions
        if u in kept_users
    ]
    print(f"Subsampled: kept {num_keep}/{len(all_users)} users, "
          f"{len(subsampled)}/{len(interactions)} interactions")

    return subsampled
```

```python
def load_and_prepare(data_dir, dataset="100k", rating_threshold=0.0,
                     min_user_interactions=5, min_item_interactions=0,
                     subsample_frac=1.0):
    """
    Full pipeline: load -> reindex -> binarize -> filter -> split -> build matrices.

    Args:
        data_dir: Path to data/ directory.
        dataset: "100k", "1m", or "10m".
        rating_threshold: Min rating to count as positive.
        min_user_interactions: Drop users with fewer interactions (default 5).
        min_item_interactions: Drop items with fewer interactions (default 0).
        subsample_frac: Fraction of users to keep (1.0 = all).

    Returns:
        dict with keys:
            train_matrix, test_dict, num_users, num_items,
            user_map, item_map, full_matrix
    """
    raw = load_movielens_raw(data_dir, dataset)
    reindexed, user_map, item_map, num_users, num_items = reindex_ids(raw)
    binarized = binarize(reindexed, threshold=rating_threshold)

    if min_user_interactions > 0 or min_item_interactions > 0:
        binarized = filter_min_interactions(
            binarized, min_user=min_user_interactions,
            min_item=min_item_interactions
        )
        # Re-reindex after filtering
        binarized, user_map_filt, item_map_filt, num_users, num_items = reindex_ids(
            binarized
        )
        inv_user = {v: k for k, v in user_map.items()}
        inv_item = {v: k for k, v in item_map.items()}
        user_map = {inv_user[k]: v for k, v in user_map_filt.items()}
        item_map = {inv_item[k]: v for k, v in item_map_filt.items()}

    if subsample_frac < 1.0:
        binarized = subsample(binarized, subsample_frac)
        # Re-reindex after subsampling
        binarized, user_map_sub, item_map_sub, num_users, num_items = reindex_ids(
            [(u, i, v, ts) for u, i, v, ts in binarized]
        )
        # Compose maps
        inv_user = {v: k for k, v in user_map.items()}
        inv_item = {v: k for k, v in item_map.items()}
        user_map = {inv_user[k]: v for k, v in user_map_sub.items()}
        item_map = {inv_item[k]: v for k, v in item_map_sub.items()}

    full_matrix = build_interaction_matrix(binarized, num_users, num_items)
    train_matrix, test_dict = leave_one_out_split(binarized, num_users, num_items)

    return {
        "train_matrix": train_matrix,
        "test_dict": test_dict,
        "full_matrix": full_matrix,
        "num_users": num_users,
        "num_items": num_items,
        "user_map": user_map,
```

```python
        "item_map": item_map,
    }


if __name__ == "__main__":
    # Quick sanity check
    script_dir = os.path.dirname(os.path.abspath(__file__))
    data_dir = os.path.join(script_dir, "..", "data")

    result = load_and_prepare(data_dir, dataset="100k")

    print(f"\n--- Summary ---")
    print(f"Users: {result['num_users']}")
    print(f"Items: {result['num_items']}")
    print(f"Train nnz: {result['train_matrix'].nnz}")
    print(f"Test users: {len(result['test_dict'])}")
    print(f"Full matrix density: "
          f"{result['full_matrix'].nnz / (result['num_users'] * result['num_items']) * 100:.4f}%")

    # Verify test items are not in training
    train = result["train_matrix"]
    errors = 0
    for u, i in result["test_dict"].items():
        if train[u, i] != 0:
            errors += 1
    print(f"Leakage check: {errors} test items found in train (should be 0)")
```

A.2 eals_numpy.py

```python
"""
Element-wise ALS (eALS) for Implicit Feedback -- NumPy reference implementation.

Based on: "Fast Matrix Factorization for Online Recommendation with Implicit Feedback"
by He et al. (2016).

Key insight: by precomputing S^q = Q^T Q (a small k×k matrix), the cost of each
element-wise update drops from O(N*k) to O(|I_u|*k + k^2), avoiding iteration
over all missing entries.

Notation:
    R          : user-item interaction matrix (M users × N items), binary
    P (M × k)  : user latent factor matrix
    Q (N × k)  : item latent factor matrix
    c_ui       : confidence for observed interactions = 1 + alpha
    c_0        : confidence for missing (unobserved) entries (uniform weight)
    S^q = Q^T Q : precomputed k×k matrix for efficient missing-data handling
"""

import time
import numpy as np
from scipy import sparse


def eals_train(train_matrix, num_factors=64, num_iter=50,
               reg=0.01, alpha=1.0, c0=1.0, seed=42, verbose=True):
    """
    Train eALS model on a sparse implicit feedback matrix.

    Args:
        train_matrix: scipy.sparse.csr_matrix (num_users × num_items), binary.
```

```
        num_factors: Latent dimension k.
        num_iter: Number of ALS iterations.
        reg: L2 regularization lambda.
        alpha: Confidence boost for observed interactions (c_ui = 1 + alpha).
        c0: Uniform confidence weight for missing entries.
        seed: Random seed.
        verbose: Print progress.

    Returns:
        P: np.ndarray (num_users × k) -- user factors.
        Q: np.ndarray (num_items × k) -- item factors.
        losses: list of training loss per iteration.
    """
    rng = np.random.RandomState(seed)
    M, N = train_matrix.shape
    k = num_factors

    # Initialize factors with small random values
    P = rng.normal(0, 0.01, (M, k))
    Q = rng.normal(0, 0.01, (N, k))

    # Confidence for observed entries
    c_obs = 1.0 + alpha  # since all r_ui = 1 after binarization

    # Precompute per-user and per-item interaction lists (CSR/CSC)
    R_csr = train_matrix.tocsr()
    R_csc = train_matrix.tocsc()

    losses = []

    for iteration in range(num_iter):
        t_start = time.time()

        # --- Update user factors P ---
        Sq = Q.T @ Q  # k × k
        _update_users(P, Q, R_csr, Sq, c_obs, c0, reg, M, k)

        # --- Update item factors Q ---
        Sp = P.T @ P  # k × k
        _update_items(P, Q, R_csc, Sp, c_obs, c0, reg, N, k)

        elapsed = time.time() - t_start

        loss = _compute_loss(P, Q, R_csr, c_obs, c0, reg, M)
        losses.append(loss)
        if verbose:
            print(f"Iteration {iteration + 1}/{num_iter} | "
                  f"loss={loss:.4f} | time={elapsed:.2f}s")

    return P, Q, losses


def _update_users(P, Q, R_csr, Sq, c_obs, c0, reg, M, k):
    """
    Update all user factors element-wise.

    For user u, factor f, the closed-form update is:

        p_uf = numerator / denominator

    numerator = sum_{i in I_u} (c_ui - c0) * q_if * hat_r_ui_f
                - c0 * (p_u^T @ Sq[:, f] - p_uf * Sq[f, f])

```

```
    where hat_r_ui_f = 1 - sum_{f'!=f} p_uf' * q_if'
                     = 1 - (p_u @ q_i - p_uf * q_if)

    denominator = sum_{i in I_u} (c_ui - c0) * q_if^2
                  + c0 * Sq[f, f] + lambda
    """
    for u in range(M):
        # Items that user u interacted with
        item_indices = R_csr[u].indices

        if len(item_indices) == 0:
            continue

        # Q submatrix for observed items: |I_u| × k
        Q_u = Q[item_indices]

        # Precompute prediction cache: p_u @ q_i for each observed item
        pred_cache = Q_u @ P[u]  # |I_u|

        for f in range(k):
            # Partial residual: remove contribution of factor f
            # hat_r = 1 - (pred - p_uf * q_if) = 1 - pred + p_uf * q_if
            q_f = Q_u[:, f]  # |I_u|
            hat_r = 1.0 - pred_cache + P[u, f] * q_f

            # Numerator
            numer = (c_obs - c0) * np.dot(q_f, hat_r)
            # Contribution from all items via precomputed Sq
            numer -= c0 * (P[u] @ Sq[:, f] - P[u, f] * Sq[f, f])

            # Denominator
            denom = (c_obs - c0) * np.dot(q_f, q_f) + c0 * Sq[f, f] + reg

            # Update
            old_val = P[u, f]
            P[u, f] = numer / denom

            # Update prediction cache to reflect the change
            pred_cache += (P[u, f] - old_val) * q_f


def _update_items(P, Q, R_csc, Sp, c_obs, c0, reg, N, k):
    """
    Update all item factors element-wise (symmetric to user update).
    """
    for i in range(N):
        # Users who interacted with item i
        user_indices = R_csc[:, i].indices

        if len(user_indices) == 0:
            continue

        P_i = P[user_indices]
        pred_cache = P_i @ Q[i]

        for f in range(k):
            p_f = P_i[:, f]
            hat_r = 1.0 - pred_cache + Q[i, f] * p_f

            numer = (c_obs - c0) * np.dot(p_f, hat_r)
            numer -= c0 * (Q[i] @ Sp[:, f] - Q[i, f] * Sp[f, f])

            denom = (c_obs - c0) * np.dot(p_f, p_f) + c0 * Sp[f, f] + reg
```

```python

            old_val = Q[i, f]
            Q[i, f] = numer / denom

            pred_cache += (Q[i, f] - old_val) * p_f


def _compute_loss(P, Q, R_csr, c_obs, c0, reg, M):
    """
    Compute the weighted squared loss:
        L = sum_u sum_i c_ui * (r_ui - p_u^T q_i)^2 + lambda*(||P||^2 + ||Q||^2)

    Uses the decomposition trick to avoid iterating over all (u,i) pairs:
        sum_u sum_i c0 * (p_u^T q_i)^2 = c0 * ||P^T Q||_F^2 = c0 * trace(Sp @ Sq
            )

    Then adjust for observed entries which have different weight and target.
    """
    Sp = P.T @ P  # k × k
    Sq = Q.T @ Q  # k × k

    # Loss from all entries assuming r=0 and weight=c0
    loss = c0 * np.trace(Sp @ Sq)

    # Correct for observed entries
    for u in range(M):
        items = R_csr[u].indices
        if len(items) == 0:
            continue

        Q_u = Q[items]
        preds = Q_u @ P[u]  # predictions for observed items

        # Remove c0 * pred^2 contribution (already counted above)
        loss -= c0 * np.dot(preds, preds)
        # Add correct contribution: c_obs * (1 - pred)^2
        residuals = 1.0 - preds
        loss += c_obs * np.dot(residuals, residuals)

    # Regularization
    loss += reg * (np.sum(P ** 2) + np.sum(Q ** 2))

    return loss


if __name__ == "__main__":
    import os
    import sys
    sys.path.insert(0, os.path.dirname(__file__))
    from data_loader import load_and_prepare

    data_dir = os.path.join(os.path.dirname(__file__), "..", "data")
    data = load_and_prepare(data_dir, dataset="100k")

    print("\nTraining eALS (NumPy)...")
    P, Q, losses = eals_train(
        data["train_matrix"],
        num_factors=32,
        num_iter=10,
        reg=0.01,
        alpha=1.0,
        c0=1.0,
    )
```

```python

    print(f"\nFinal loss: {losses[-1]:.4f}")
    print(f"P shape: {P.shape}, Q shape: {Q.shape}")
```

A.3 eals_rdd.py

```python
"""
Element-wise ALS (eALS) for Implicit Feedback -- PySpark RDD implementation.

Parallelization strategy (from the paper, Section 4):
    - When updating P (user factors): partition users across workers.
      Broadcast Q and S^q = Q^T Q. Each worker independently updates
      its assigned users' factor vectors.
    - When updating Q (item factors): partition items across workers.
      Broadcast P and S^p = P^T P. Each worker independently updates
      its assigned items' factor vectors.

This is embarrassingly parallel per half-step because each user's (or item's)
update only depends on the *current* opposite factor matrix, not on other
users (or items) being updated in the same half-step.
"""

import time
import numpy as np
from scipy import sparse


def eals_train_rdd(sc, train_matrix, num_factors=64, num_iter=50,
                   reg=0.01, alpha=1.0, c0=1.0, num_partitions=None,
                   seed=42, verbose=True):
    """
    Train eALS model using PySpark RDDs.

    Args:
        sc: SparkContext.
        train_matrix: scipy.sparse.csr_matrix (num_users × num_items), binary.
        num_factors: Latent dimension k.
        num_iter: Number of ALS iterations.
        reg: L2 regularization lambda.
        alpha: Confidence boost for observed entries (c_ui = 1 + alpha).
        c0: Uniform confidence weight for missing entries.
        num_partitions: Number of RDD partitions (default: sc.defaultParallelism
            ).
        seed: Random seed.
        verbose: Print progress.

    Returns:
        P: np.ndarray (num_users × k).
        Q: np.ndarray (num_items × k).
        losses: list of loss per iteration.
    """
    rng = np.random.RandomState(seed)
    M, N = train_matrix.shape
    k = num_factors

    if num_partitions is None:
        num_partitions = sc.defaultParallelism

    c_obs = 1.0 + alpha

    # Initialize factors
    P = rng.normal(0, 0.01, (M, k))
```

```python
    Q = rng.normal(0, 0.01, (N, k))

    # Build per-user interaction data: list of (user_id, item_indices_array)
    R_csr = train_matrix.tocsr()
    user_data = []
    for u in range(M):
        items = R_csr[u].indices.copy()
        if len(items) > 0:
            user_data.append((u, items))

    # Build per-item interaction data
    R_csc = train_matrix.tocsc()
    item_data = []
    for i in range(N):
        users = R_csc[:, i].indices.copy()
        if len(users) > 0:
            item_data.append((i, users))

    # Create and cache RDDs (static structure, reused every iteration)
    user_rdd = sc.parallelize(user_data, num_partitions).cache()
    item_rdd = sc.parallelize(item_data, num_partitions).cache()

    losses = []

    for iteration in range(num_iter):
        t_start = time.time()

        # ---- Update P (user factors) ----
        Sq = Q.T @ Q  # k × k
        Q_bc = sc.broadcast(Q)
        Sq_bc = sc.broadcast(Sq)

        def update_user_partition(iterator):
            """Process a partition of users, updating all their factors."""
            q_local = Q_bc.value
            sq_local = Sq_bc.value
            results = []

            for u, item_indices in iterator:
                p_u = P_bc.value[u].copy()
                Q_u = q_local[item_indices]  # |I_u| × k
                pred_cache = Q_u @ p_u

                for f in range(k):
                    q_f = Q_u[:, f]
                    hat_r = 1.0 - pred_cache + p_u[f] * q_f

                    numer = (c_obs - c0) * np.dot(q_f, hat_r)
                    numer -= c0 * (p_u @ sq_local[:, f] - p_u[f] * sq_local[f, f])

                    denom = (c_obs - c0) * np.dot(q_f, q_f) + c0 * sq_local[f, f] + reg

                    old_val = p_u[f]
                    p_u[f] = numer / denom
                    pred_cache += (p_u[f] - old_val) * q_f

                results.append((u, p_u))
            return iter(results)

        P_bc = sc.broadcast(P)
        updated_users = user_rdd.mapPartitions(update_user_partition).collect()
```

```
        for u, p_u in updated_users:
            P[u] = p_u

        Q_bc.unpersist()
        Sq_bc.unpersist()
        P_bc.unpersist()

        # ---- Update Q (item factors) ----
        Sp = P.T @ P  # k × k
        P_bc = sc.broadcast(P)
        Sp_bc = sc.broadcast(Sp)

        def update_item_partition(iterator):
            """Process a partition of items, updating all their factors."""
            p_local = P_bc.value
            sp_local = Sp_bc.value
            results = []

            for i, user_indices in iterator:
                q_i = Q_bc2.value[i].copy()
                P_i = p_local[user_indices]
                pred_cache = P_i @ q_i

                for f in range(k):
                    p_f = P_i[:, f]
                    hat_r = 1.0 - pred_cache + q_i[f] * p_f

                    numer = (c_obs - c0) * np.dot(p_f, hat_r)
                    numer -= c0 * (q_i @ sp_local[:, f] - q_i[f] * sp_local[f, f
                        ])

                    denom = (c_obs - c0) * np.dot(p_f, p_f) + c0 * sp_local[f, f
                        ] + reg

                    old_val = q_i[f]
                    q_i[f] = numer / denom
                    pred_cache += (q_i[f] - old_val) * p_f

                results.append((i, q_i))
            return iter(results)

        Q_bc2 = sc.broadcast(Q)
        updated_items = item_rdd.mapPartitions(update_item_partition).collect()
        for i, q_i in updated_items:
            Q[i] = q_i

        P_bc.unpersist()
        Sp_bc.unpersist()
        Q_bc2.unpersist()

        elapsed = time.time() - t_start

        loss = _compute_loss_fast(P, Q, R_csr, c_obs, c0, reg)
        losses.append(loss)
        if verbose:
            print(f"Iteration {iteration + 1}/{num_iter} | "
                  f"loss={loss:.4f} | time={elapsed:.2f}s")

    # Clean up cached RDDs
    user_rdd.unpersist()
    item_rdd.unpersist()

    return P, Q, losses
```

```python


def _compute_loss_fast(P, Q, R_csr, c_obs, c0, reg):
    """
    Compute weighted squared loss using the trace trick to avoid O(M*N) cost.

    L = c0 * trace(P^T P @ Q^T Q)                          [all-pairs, r=0 assumed]
      - c0 * sum_{(u,i) observed} (p_u^T q_i)^2            [remove wrong c0 term]
      + c_obs * sum_{(u,i) observed} (1 - p_u^T q_i)^2     [add correct observed term]
      + reg * (||P||^2 + ||Q||^2)
    """
    M = P.shape[0]
    Sp = P.T @ P
    Sq = Q.T @ Q

    loss = c0 * np.trace(Sp @ Sq)

    for u in range(M):
        items = R_csr[u].indices
        if len(items) == 0:
            continue
        preds = Q[items] @ P[u]
        loss -= c0 * np.dot(preds, preds)
        residuals = 1.0 - preds
        loss += c_obs * np.dot(residuals, residuals)

    loss += reg * (np.sum(P ** 2) + np.sum(Q ** 2))
    return loss


if __name__ == "__main__":
    import os
    import sys
    from pyspark import SparkContext, SparkConf

    sys.path.insert(0, os.path.dirname(__file__))
    from data_loader import load_and_prepare

    data_dir = os.path.join(os.path.dirname(__file__), "..", "data")
    data = load_and_prepare(data_dir, dataset="100k")

    conf = SparkConf().setAppName("eALS-RDD").setMaster("local[*]")
    sc = SparkContext(conf=conf)
    sc.setLogLevel("WARN")

    print("\nTraining eALS (PySpark RDD)...")
    P, Q, losses = eals_train_rdd(
        sc,
        data["train_matrix"],
        num_factors=32,
        num_iter=10,
        reg=0.01,
        alpha=1.0,
        c0=1.0,
    )

    print(f"\nFinal loss: {losses[-1]:.4f}")
    print(f"P shape: {P.shape}, Q shape: {Q.shape}")

    sc.stop()
```

A.4 evaluation.py

```python
"""
Evaluation metrics for implicit feedback recommendation.

Uses the leave-one-out protocol from the eALS paper:
    For each test user, rank the held-out item against all items the user
    has NOT interacted with in training. Compute Hit Rate and NDCG at K.
"""

import numpy as np


def evaluate(P, Q, train_matrix, test_dict, top_k=10):
    """
    Evaluate recommendation quality using Hit Rate @ K and NDCG @ K.

    Args:
        P: np.ndarray (num_users × k) -- user factors.
        Q: np.ndarray (num_items × k) -- item factors.
        train_matrix: scipy.sparse.csr_matrix -- training interactions.
        test_dict: dict {user_id: test_item_id} -- one held-out item per user.
        top_k: Cutoff K for ranking metrics.

    Returns:
        dict with "HR@K" and "NDCG@K".
    """
    hits = 0
    ndcg_sum = 0.0
    num_test_users = len(test_dict)

    R_csr = train_matrix.tocsr()
    num_items = Q.shape[0]

    for u, test_item in test_dict.items():
        # Score all items for this user
        scores = Q @ P[u]  # (num_items,)

        # Mask out training items (set to -inf so they don't appear in top-K)
        train_items = R_csr[u].indices
        scores[train_items] = -np.inf

        # Get top-K item indices
        top_k_items = np.argpartition(scores, -top_k)[-top_k:]
        top_k_items = top_k_items[np.argsort(-scores[top_k_items])]

        # Check if test item is in top-K
        if test_item in top_k_items:
            hits += 1
            # NDCG: position of test item in the ranked list (1-indexed)
            rank = np.where(top_k_items == test_item)[0][0] + 1
            ndcg_sum += 1.0 / np.log2(rank + 1)

    hr = hits / num_test_users
    ndcg = ndcg_sum / num_test_users

    return {f"HR@{top_k}": hr, f"NDCG@{top_k}": ndcg}


def evaluate_multiple_k(P, Q, train_matrix, test_dict, k_values=(5, 10, 20)):
    """
    Evaluate at multiple K values in a single pass.

```

```python
    Returns:
        dict mapping metric names to values, e.g. {"HR@5": 0.3, "NDCG@5": 0.15, ...}
    """
    max_k = max(k_values)
    results = {f"HR@{k}": 0.0 for k in k_values}
    results.update({f"NDCG@{k}": 0.0 for k in k_values})

    R_csr = train_matrix.tocsr()
    num_test_users = len(test_dict)

    for u, test_item in test_dict.items():
        scores = Q @ P[u]
        train_items = R_csr[u].indices
        scores[train_items] = -np.inf

        top_items = np.argpartition(scores, -max_k)[-max_k:]
        top_items = top_items[np.argsort(-scores[top_items])]

        if test_item in top_items:
            rank = np.where(top_items == test_item)[0][0] + 1
            for k in k_values:
                if rank <= k:
                    results[f"HR@{k}"] += 1.0
                    results[f"NDCG@{k}"] += 1.0 / np.log2(rank + 1)

    for key in results:
        results[key] /= num_test_users

    return results


if __name__ == "__main__":
    import os
    import sys
    sys.path.insert(0, os.path.dirname(__file__))
    from data_loader import load_and_prepare
    from eals_numpy import eals_train

    data_dir = os.path.join(os.path.dirname(__file__), "..", "data")
    data = load_and_prepare(data_dir, dataset="100k")

    print("\nTraining eALS for evaluation demo...")
    P, Q, losses = eals_train(
        data["train_matrix"],
        num_factors=32,
        num_iter=20,
        reg=0.01,
        alpha=1.0,
        c0=1.0,
    )

    print("\nEvaluating...")
    metrics = evaluate_multiple_k(
        P, Q, data["train_matrix"], data["test_dict"],
        k_values=(5, 10, 20)
    )
    for name, val in sorted(metrics.items()):
        print(f"  {name}: {val:.4f}")
```